YouTube Comment Intelligence

Put video links in. Get a reception report out. Works on brand ads, creator reviews, and explainer or viral content.

Answers Which ideas from a video reached the audience, which did not, and what people talked about instead

In YouTube URLs, two free API keys

Out `report.pdf`, `comments.csv`, `summary.csv`. Three files, nothing else

Stack Python 3.10+, YouTube Data API v3, Gemini API, HuggingFace transformers (local), pandas

Cost Zero. Four to five model calls per run regardless of corpus size

STRATEGY AND CREATIVE

Sections 1 and 5 are enough to brief the tool and read its output. Skip the code.

The thing to internalise: a low score means an idea *did not arrive*, which is a different diagnosis from being rejected. The fix for the first is execution and distribution. The fix for the second is the idea.

DATA AND ANALYTICS

Section 3 is the part worth arguing with. The architecture inverts the obvious LLM-labelling design for cost, reproducibility and auditability, and that section states the tradeoff honestly, including what it gives up.

Section 2 covers the API-efficiency choices.

THE CORE IDEA

The pipeline reads the video and the comments **separately**, then checks one against the other. What the video pushed comes from the transcript. What the audience said comes from the comments. Neither analysis sees the other. Only at the end does it ask whether the first shows up in the second.

Keeping them apart is what makes the answer mean anything. If the model saw the comments while describing the video, "did the message land" would always answer yes.

SECTION 1

What goes in

One config file, two keys, any number of links

```
VIDEOS = [
    {"url": "https://www.youtube.com/watch?v=abc123",
     "group": "Campaign A", "kind": "brand_ad"},
    {"url": "https://www.youtube.com/shorts/def456",
     "group": "Campaign A", "kind": "review"},
    {"url": "https://youtu.be/ghi789",
     "group": "Campaign B", "kind": "explainer"},
]
```

FIELD	DOES	NOTES
`url`	The video	Accepts `/watch?v=`, `/shorts/` and `youtu.be/`. Shorts work identically: YouTube draws no API-level distinction
`group`	The comparison unit	Videos sharing a group are pooled and treated as one campaign. This is what the report compares, and what one briefing call covers
`kind`	Selects the analysis lens	`brand_ad` looks for creative decisions. `review` for which features the creator foregrounded. `explainer` for claims and topics. `auto` lets the model decide

The two keys

Both free, both Google, from different places. This catches people out.

KEY	WHERE	QUOTA
YouTube Data API v3	Google Cloud Console. Enable the API in the library, then create a key under Credentials	10,000 units/day. A page of 100 comments costs 1 unit
Gemini API	Google AI Studio at `aistudio.google.com`, *not* the Cloud Console	Free tier, no card. Four to five calls per run

TWO THINGS THAT WILL BITE YOU

Whitespace in a pasted key. A tab or newline inside the key produces `httpx.InvalidURL: Invalid non-printable ASCII character in URL`, because the key is concatenated into the endpoint. Check with `python -c "import config; print(repr(config.GEMINI_API_KEY))"` and look for `\t`.

Model IDs expire. `MODEL_CHEAP` and `MODEL_SMART` are config variables because Google retires free-tier models every few months. A 404 means the ID is stale, not that the code is broken.

Settings

SETTING	DEFAULT	EFFECT
`SESSION_NAME`	blank	Output folder name. Blank derives it from the group names
`KEEP_LANGUAGES`	`{id, ms, en, tl}`	Whitelist. `None` keeps everything. `tl` is in there because langdetect often tags Indonesian slang as Tagalog
`MIN_COMMENT_LETTERS`	8	Measured after stripping punctuation and emoji
`CODEBOOK_SAMPLE_SIZE`	150	Floor. Actual is `max(150, 8% of corpus)`, capped at `CODEBOOK_SAMPLE_MAX`
`UNCLASSIFIED_LIMIT`	30	Above this share, one extra call extends the codebook
`EMOTION_MODE`	`"emotion"`	`"emotion"` or `"sentiment"`. Always runs
`CAMPAIGN_BACKGROUND`	`True`	Generate the ungrounded background. See section 4
`REPORT_LANGUAGE`	`"English"`	Prose language. Comments stay in their original language
`KEEP_INTERMEDIATE`	`False`	Adds `debug/`. See section 5

PREFLIGHT

Before any call, `run.py` checks that both keys are set, `VIDEOS` is populated, `EMOTION_MODE` is valid, `transformers` is importable, and a PDF engine exists. Problems are reported as one list. Without this, a missing PDF engine would fail at the last step, after five model calls and a 500 MB download had already been paid for.

It can only see the Python package, not the browser that `playwright install chromium` downloads. If preflight passes and rendering then fails, that missing browser is why.

SECTION 2

How it flows

Five stages, two of which cost tokens

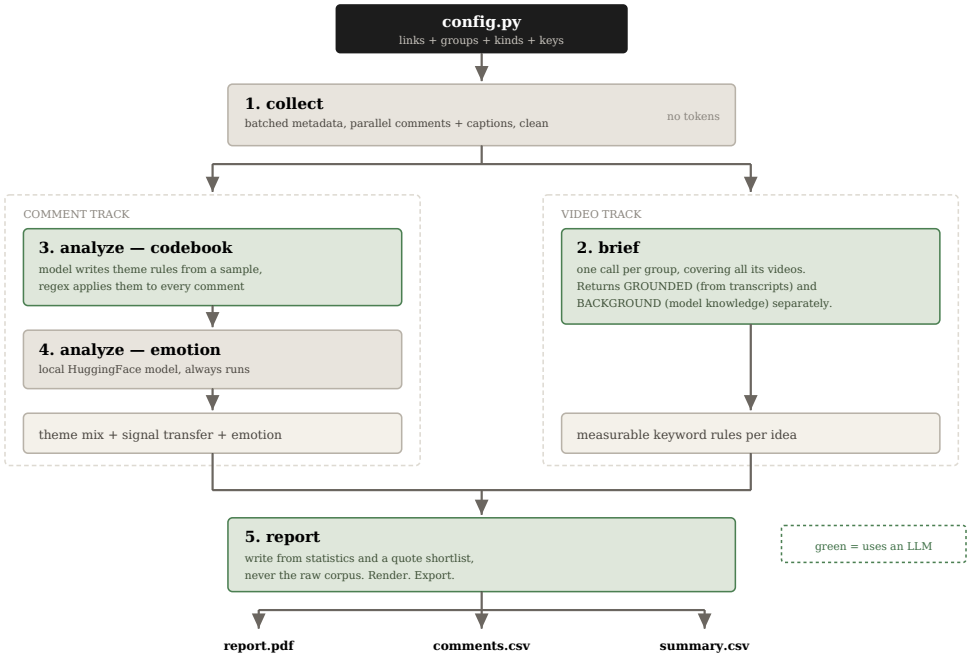

Figure 1. The two tracks never meet until stage 5.

#	MODULE	LLM CALLS	PRODUCES
1	`collect.py`	0	Comments with replies, metadata, captions, then cleaned and language-tagged
2	`brief.py`	1 per group	Grounded brief, ungrounded background, and 4 to 8 measurable points per video
3	`analyze.py`	1, sometimes 2	Codebook applied to the corpus, theme mix and signal transfer tables
4	`analyze.py`	0 (local)	Per-comment emotion label and confidence
5	`report.py`	1	The three output files

Where the time goes, and what was done about it

WAS	NOW	WHY
One `videos.list` call per video	One batched call	The endpoint accepts 50 comma-separated IDs
Comments and captions fetched serially	Concurrent, 4 workers	Both are I/O bound. The pool is deliberately small: YouTube throttles bursts
One brief per video, plus one background per group	One call per group covering both	9 calls to 3 on a six-video, three-group run. Each call still sees one campaign only, so nothing is diluted

Concurrency, and the bug it caused

Parallelising the fetch introduced a real failure worth recording, since anyone extending `collect.py` can reintroduce it in one line. `googleapiclient` sits on `httplib2`, which is **not thread-safe**. One service object shared across four workers has them interleaving reads on a single socket, and it does not fail cleanly: it surfaces as `[SSL] record layer failure` or `AttributeError: 'NoneType' object has no attribute 'read'` from inside `http.client`, which looks like a problem with the video rather than with the client.

Each thread now builds its own service through `collect._service()`, backed by `threading.local()`. `cache_discovery=False` disables the on-disk discovery cache, a second piece of shared state.

FAILURE	RETRIED?	WHY
Socket or SSL error	3 attempts, backoff	Transient. The thread's client is discarded before each retry: a broken connection stays broken
`HttpError` 403 / 404	No	Comments disabled, or the video is gone. Neither improves on a second attempt
A video exhausting its retries	No	Reports, keeps what it collected, continues. One bad video should not lose the other five
Every video failing	No	Raises with a plain message rather than letting an empty frame fail three stages later

SECTION 3

Why the model writes rules, not labels

The design decision worth arguing with

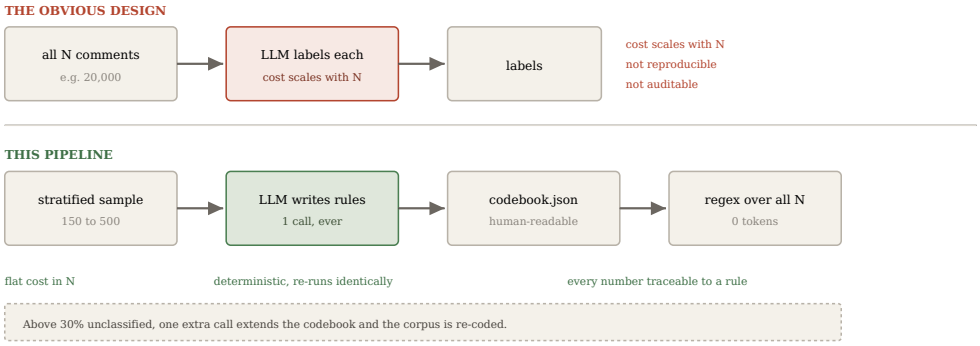

Figure 2. The model does the judgement. The code does the counting.

What it buys

- **Flat cost.** You pay for the sample, not the corpus. 200 comments and 20,000 cost the same.
- **Reproducibility.** Regex is deterministic. An LLM labelling 900 comments twice will not agree with itself, which makes before-and-after comparison meaningless.
- **Auditability.** `debug/codebook.json` holds the exact rules behind every figure.
- **Portability.** The vocabulary is derived from whichever conversation you point it at, so the same code works on fintech, music or politics with no edits.

What it costs

- **Paraphrase.** Praise that uses none of the keywords is missed. Transfer figures say whether a subject *arrived*, not its precise share.
- **Context.** Regex cannot read negation or sarcasm. "Not worth it" matches a `worth` keyword.
- **Rare themes.** Something present in well under 1% of a large corpus can be missed by the sample, and rare is not the same as unimportant.

> **AN OPTIMISATION THAT DID NOT WORK**
>
> Theme application was rewritten from a row-wise `.apply()` to a vectorised `str.contains()`, expecting a speedup. On 20,000 comments it came out **0.6x, slower**: pandas still runs Python's `re` per element, so there is nothing to vectorise. Output was verified byte-identical and the clearer version kept, but it is not a performance win and is not claimed as one.

> **WHY THIS MATTERS, CONCRETELY**
>
> An early version measured a Honda campaign's concept-car heritage hook at 7.5% transfer. The codebook showed that nine of those ten matches were the brand's own dealer account posting the trim name, which was in the keyword list. The real figure was under 1%.
>
> The briefing prompt now forbids the brand or product name in any keyword, and says why. But the error was findable in the first place only because the rule was written to a file a human could read. That is the argument for this architecture in one example.

On the sample

The sample never produces a number. It only *discovers* themes and vocabulary, so it does not need to be proportionally representative. It needs one example of everything worth naming, which is a far easier bar. Estimating "what share of this thread is about price" from 150 of 20,000 would be a real problem. Naming price as a theme needs one clear instance.

It is not a flat random draw, which would over-represent whichever video got the most comments and miss the high-engagement comments where shared vocabulary lives. Per group it takes the most-liked, the longest, and a random remainder. Size is `max(150, 8% of corpus)` capped at 500, because a theme at 5% appears in a 150-draw essentially every time and one at 1% needs a few hundred.

SECTION 4

Two briefs, kept apart

What keeps the report honest

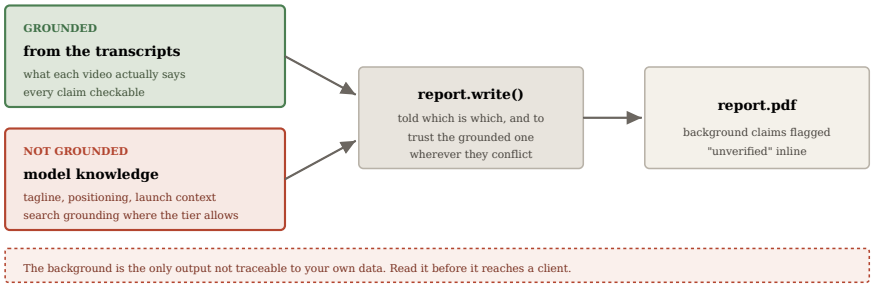

Figure 3. Both reach the report. Only one is evidence.

A transcript will not tell you who a campaign was aimed at, what the tagline was, or how many units were capped. That context is useful and a strategist would normally spend a morning finding it, so the pipeline asks the model for it.

But a model asked "what was campaign X about" produces fluent, specific, confident detail whether or not it knows, and taglines, unit counts, launch dates and prices are exactly what it invents. Search grounding is enabled where the tier supports it so claims return with source URLs, and the prompt requires anything inferred to be marked unverified. Grounding is metered separately and is not on every tier; the call degrades to a plain one automatically rather than failing.

Guardrails in the writing prompt

The model that writes the report is given the statistics, not the corpus, and is instructed to:

- use only the numbers supplied, and never invent one
- quote only from a supplied shortlist, verbatim, with any translation in a gloss line underneath rather than inside the quote
- use only the closed verdict vocabulary: `Yes`, `Partly`, `Barely`, `No`, `Backfired`, `Not used`, `Loud`
- name any group under 100 comments as unreliable in that group's own section, not just in the limitations
- flag background-derived claims inline
- treat low transfer as "did not arrive" rather than "was rejected"

These exist because a model handed a pile of comments will produce confident percentages that are not in the data.

SECTION 5

What comes out

Three files per session

```
output/
  kia-sonet-carens-rivals/        <- session A
      report.pdf
      comments.csv
      summary.csv
  converse-campaign/              <- session B, unrelated
      ...
```

Each run gets its own folder, named from `SESSION_NAME` or, if blank, from the group names. Repeat runs get `-2`, `-3` rather than overwriting.

report.pdf

An internal debrief, not a client deliverable: dense type, no cover page, no methodology section. Per group it gives the background in two or three sentences, a verdict table, what filled the conversation instead, two or three verbatim quotes with English glosses, and exactly two actions. Then a cross-group section, the one thing to carry forward, and a short limitations block.

Emotion is folded into each group's header line rather than given a section of its own. It is the weakest evidence in the pipeline and a section would give it more weight than it earns.

> **The PDF is required.** Preflight refuses to start without an engine, so a missing one costs a second rather than five model calls. HTML is a build artifact written to a temp file and deleted; turn on `KEEP_INTERMEDIATE` to keep it and the raw markdown in `debug/`.

PDF engines

One is required. They are tried in this order.

ENGINE	NOTES
`playwright>=1.49`	Recommended. Best output and identical on every OS. Two commands: `pip install playwright` then `playwright install chromium`
`weasyprint>=68`	Needs GTK on Windows, which is the painful part. 68 is the floor because CVE-2025-68616 was fixed there. Recent versions also require a newer Python
`pdfkit>=1.0.0`	A wrapper only; the wkhtmltopdf binary installs separately. Last release was 2021 and wkhtmltopdf is archived upstream. Last resort on a machine that already has it

comments.csv

One row per comment: group, video, text, theme, ideas echoed, emotion and confidence, likes, language, and the topic-neutral flags. Sorted by group then likes, so the comments a thread rallied around sit at the top of each block. This is the file for handpicking quotes.

summary.csv

Tidy long format, one row per number.

```
group,metric,label,value,unit,n
Honda BR-V N7X,base,comments analysed,110,count,110
Honda BR-V N7X,theme,Price pushback,7.3,percent,8
Honda BR-V N7X,signal_transfer,Hero colour,10.4,percent,11
Honda BR-V N7X,emotion,happy,62.1,percent,68
```

Long rather than wide because it pivots without reshaping. Filter on `metric` to get the rows behind one chart and drop it into Sheets. The pipeline draws no charts itself: this file exists so the design team builds their own.

debug/

`KEEP_INTERMEDIATE = True` adds the raw fetch, the briefs, the background, the codebook, and the report markdown and HTML. **No stage reads any of these:** data passes between stages in memory, so they exist purely for auditing. The one worth turning on when a number looks wrong is `codebook.json`.

Reading order for a sceptical reader

1. `debug/codebook.json`. Are the rules sensible? Does any keyword contain a brand or product name?
2. `summary.csv` filtered to `signal_transfer`, then `comments.csv` filtered on `echoed_ideas` to read the comments behind a figure.
3. `debug/campaign_background.md`. Anything asserted that you cannot verify?
4. `report.pdf`.

Known limits

LIMIT	CONSEQUENCE
Keyword matching undercounts paraphrase	Read transfer as arrived or did not arrive, not a precise share
Emotion labels have no context	Sarcasm and measured criticism both read as anger. Directional only, and the confidence share travels with the numbers
No captions on a video	Brief falls back to title and description, flagged in the output

Groups under about 100 comments	Not reliable. The report is instructed to say so per group
Commenters are not buyers	Self-selecting. Directional qualitative input, not market research
Free-tier prompts may train the provider's models	Use a paid tier or Vertex AI for confidential client work

Common errors

SYMPTOM	CAUSE
`ModuleNotFoundError: googleapiclient`	Ran with `py` instead of `python` on Windows. The `py` launcher uses the system Python and ignores the active conda environment
`httpx.InvalidURL: Invalid non-printable ASCII character`	A tab or newline inside a pasted API key, which is concatenated into the endpoint. Check with `print(repr(config.GEMINI_API_KEY))`
404 from Gemini	Stale model ID. Update `MODEL_CHEAP` and `MODEL_SMART`
`JSONDecodeError` during briefing	Malformed JSON from the model. `llm.ask_json` retries by sending it back to be repaired; a hard failure usually means the response was truncated
`[SSL] record layer failure`	Was the thread-safety bug in section 2, since fixed. If it returns, check nothing shares a service object across threads

Running it

```
pip install -r requirements.txt
pip install playwright && playwright install chromium
# edit config.py: links and both keys
python run.py
```